AUTOMATED TIMETABLE GENERATOR

IILM UNIVERSITY

Submitted by
ADITYA RANJAN KUMAR | CS-2341901

Programme
B.Tech – CSE, Sem 7

Title
Professional UI/UX Design & Web Prototyping Intern

Organisation
Eduskills

AGENDA

// What we will cover today

01 Problem Statement

Why manual timetabling fails

02 Project Overview

What this system does

03 Key Features

Core capabilities at a glance

04 System Architecture

3-tier full-stack design

05 Scheduling Algorithm

Phase 1 (labs) + Phase 2 (theory)

06 Tech Stack

HTML · CSS · JS · Node · MongoDB

07 UI & User Workflow

3-step wizard walkthrough

08 Results & Demo

Output, stats, print, CSV, save/load

09 Future Enhancements

What comes next

10 Conclusion

Summary and impact

PROBLEM STATEMENT

// The challenge of manual academic scheduling

"Scheduling an academic timetable is an NP-hard combinatorial problem. Manual scheduling at scale leads to conflicts, errors, and wasted hours every semester."

⚡ Teacher Conflicts

Same teacher scheduled in 2 subjects simultaneously

🔬 Lab Misplacement

Lab sessions split across non-consecutive periods

📊 Workload Imbalance

Subjects unevenly distributed across the week

🔄 Rigid Regeneration

Any change requires a complete manual redo

💾 No Persistence

Schedules lost at end of session unless saved manually

⏱ Time-Consuming

Hours spent per semester on a repetitive task

PROJECT OVERVIEW

// What the Automated Timetable Generator does

What it is

A browser-based, full-stack web application that automatically generates conflict-free academic timetables for engineering institutes with a single click.

Who it's for

Department administrators, faculty coordinators, and HODs who manage semester timetables for B.Tech / engineering sections.

Project URL

http://127.0.0.1:5500/public/index.html

⚡ **Click**

to generate a full timetable

0 **Conflicts**

guaranteed by algorithm

7 **Subjects**

supported in sample dataset

3 **Steps**

configuration → subjects → generate

KEY FEATURES

// Core capabilities at a glance

3-Step Wizard

Configuration → Subjects → Generate. Guided user workflow with no learning curve.

Instant Generation

Randomised CSP algorithm produces a full conflict-free timetable in under 100 ms.

Regenerate

Each Regenerate click produces a completely different valid layout.

Colour-Coded Grid

Every subject gets a unique colour theme. Dark-themed interactive visual grid.

CSV Export

Download the full timetable as a .csv file, ready for Excel or Google Sheets.

Smart Print

Landscape dark-theme print — only the timetable, no UI clutter.

MongoDB Persistence

Save timetables to a database and reload any previously saved schedule.

Live Statistics

Subjects count, teachers, total slots, filled slots, utilisation % displayed instantly.

Subject Legend

Colour legend below the grid for quick subject identification.

SYSTEM ARCHITECTURE

// 3-Tier Full-Stack Design

PRESENTATION LAYER

- HTML5 + CSS3 + JavaScript
- Dark Theme UI (CSS Variables)
- 3-Step Tabbed Wizard
- Responsive Grid / Flexbox
- Custom @media print

APPLICATION LAYER

- Node.js Runtime
- Express.js REST API
- POST /api/timetables
- GET /api/timetables
- GET /api/timetables/:id

Client ↔ API via Fetch / AJAX

DATA LAYER

- MongoDB Database
- Timetable Document Store
- Grid Array Storage
- Subjects Array
- Timestamp & Metadata

TECHNOLOGY STACK

// Tools and frameworks used

Frontend	**HTML5 + CSS3** Semantic markup, CSS custom variables, Grid, Flexbox, @media print
Fonts	**Google Fonts** Syne (display) + Space Mono (monospace data labels)
Logic	**Vanilla JavaScript** Scheduling engine, DOM rendering, Fetch API, state management
Backend	**Node.js + Express.js** REST API server: POST/GET /api/timetables, JSON responses
Database	**MongoDB** Document store for timetable grids, subjects, timestamps
Export	**CSV + Print API** Downloadable CSV via anchor tag + window.print() with @media print CSS

USER WORKFLOW

// 3-Step Wizard Walkthrough

01 STEP 1 — Configuration

- Enter College Name and Department
- Select Year / Semester and Section
- Choose Periods Per Day (6, 7, or 8)
- Choose Working Days (5 or 6)
- Edit Period Timings (pre-filled by default)

02 STEP 2 — Subjects & Teachers

- Enter Subject Name, Code, Teacher
- Select Type: Theory / Lab / Elective / Core
- Set Periods per Week (1–10)
- Pick a Colour Theme for each subject
- Click Load Sample for instant CSE Sem 6 data

03 STEP 3 — Generate & Export

- Click Generate Timetable → instant output
- View colour-coded grid with stats & legend
- Click Regenerate for a different valid layout
- Export CSV → download for Excel / Sheets
- Print (landscape dark theme) or Save to DB

RESULTS & DEMO

// Sample output and performance metrics

7	5	42	22	52%	0
Subjects	Teachers	Total Slots	Filled Slots	Utilisation	Conflicts

Feature Testing Summary

Feature	Test Condition	Result
Timetable Generation	7 subjects, 7 periods, 6 days	Conflict-free in < 100ms
Lab Placement	CN Lab + Compiler Lab	Always consecutive periods
Teacher Conflict Check	5 teachers across 7 subjects	Zero conflicts in 20 runs
Regenerate	20 consecutive regenerations	All layouts different & valid
CSV Export	Download and open in Excel	Correct formatting
Print (Dark Theme)	Browser print preview, landscape	Only timetable, dark colours
Save to MongoDB	POST to /api/timetables	Document inserted with ID
Load Saved	GET list then GET by ID	Full timetable reloaded

FUTURE ENHANCEMENTS

// What comes next

Multi-Section Support

Generate conflict-free timetables for multiple sections simultaneously with shared teacher tracking.

Soft Constraint Handling

Teacher preferences for specific time slots using a weighted scoring system.

PDF Export

Server-side PDF generation via Puppeteer for a print-ready formatted document output.

Mobile Application

Port the frontend to React Native for on-the-go timetable access on smartphones.

Room Allocation

Add classroom and lab room as an additional scheduling constraint layer.

Genetic Algorithm Backend

Replace randomised backtracking with a GA for higher slot utilisation and global optimisation.

Role-Based Authentication

Login system for admin, teachers, and students with role-appropriate views.

Analytics Dashboard

Per-teacher workload analytics and slot distribution charts using Chart.js or D3.js.

CONCLUSION

// Summary and impact

The Automated Timetable Generator successfully demonstrates that a lightweight, browser-based full-stack application can solve the academic scheduling problem — eliminating teacher conflicts, automates lab placement, and delivering a polished, production-ready scheduling tool.

Objectives Met

All 6 stated objectives achieved: config wizard, algorithm, visual grid, CSV, print, MongoDB persistence.

Zero Conflicts

Guaranteed teacher conflict-free output and lab continuity across every single generated timetable.

Full-Stack Ready

End-to-end implementation: HTML/CSS/JS frontend + Node/Express API + MongoDB — fully deployable.

Thank You

Questions & Discussion